Report: Magic Square Assignment

Overview and Goals:

The assignment tasked us with two main objectives: creating a command-line application to generate magic squares for odd integers and developing a command-line game for reconstructing shuffled magic squares. The primary goal was to implement these functionalities in Java while ensuring clarity, usability, and adherence to specified requirements.

Solution Design:

In addressing the first part (Part A), I devised an algorithm based on the provided specifications. The algorithm begins by initializing a 2D array of size n x n and setting all values to 0. It then inserts numbers from 1 to n^2 into the array, ensuring that each row, column, and diagonal sums up to the magic constant. To achieve this, the algorithm utilizes a systematic approach to place numbers in the matrix while following specific rules. The implementation involves careful consideration of edge cases and boundary conditions to ensure correctness.

For the magic square reconstruction game (Part B), the design revolves around providing an interactive experience for the user. The game starts by shuffling a generated magic square, simulating a scenario where the user is presented with a scrambled puzzle. The user is then prompted to make moves by specifying the row, column, and direction to swap elements. The challenge lies in ensuring that the moves are valid and lead to reconstructing the original magic square. To achieve this, the game logic includes checks for boundary conditions, input validation, and feedback mechanisms to guide the user.

Completed Solution:

Upon completion, the solution encompasses the entire scope outlined in the assignment. Part A delivers an accurate and efficient magic square generator, capable of handling various input sizes and producing valid magic squares. The command-line application provides a seamless user experience, guiding the user through input prompts and displaying the generated magic square in a clear format.

Part B offers an engaging and challenging magic square reconstruction game. The implementation ensures that the game is playable and enjoyable while maintaining the integrity of the magic square concept. The user interface provides intuitive feedback, guiding the user through the gameplay process and reporting the number of moves made upon completion.

Throughout the development process, I encountered challenges such as handling user input validation and ensuring the correctness of the generated magic squares. However, by adopting a systematic approach to debugging and testing, I successfully addressed these challenges and delivered a robust solution.

Software Test Methodology:

To validate the solution's correctness and reliability, I employed a comprehensive testing approach. Unit testing was conducted using JUnit to verify the functionality of critical components, such as the magic square generator algorithm and the game logic. Test cases were designed to cover various scenarios and edge cases to ensure thorough validation.

In addition to unit testing, extensive manual testing was performed to assess the solution's behaviour under different conditions. This involved running the application with various inputs, including valid and invalid inputs, to validate its responsiveness and error-handling capabilities. The testing process focused on identifying and addressing any discrepancies or anomalies in the solution's behaviour, ensuring that it performs as intended across different scenarios.

In summary, the completed solution represents a robust and user-friendly implementation of both the magic square generator and the reconstruction game. Through diligent design, testing, and refinement, the solution meets the goals of the assignment while providing an enjoyable and interactive experience for users.